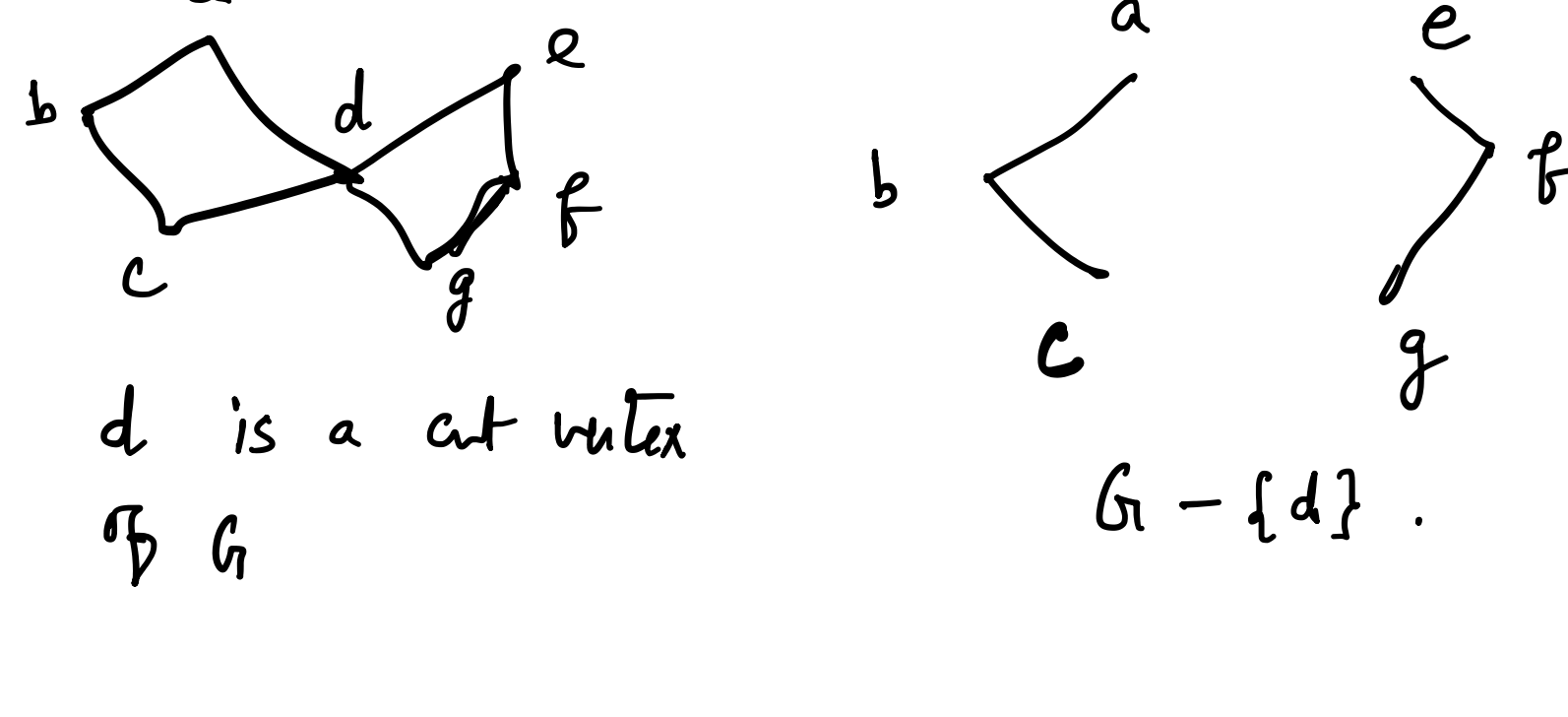


Algorithm for cut vertices:

This is a typical and canonical example of using DFS to solve graph problems.

Let G be a connected graph. A vertex v is a cut vertex if its removal disconnects G .



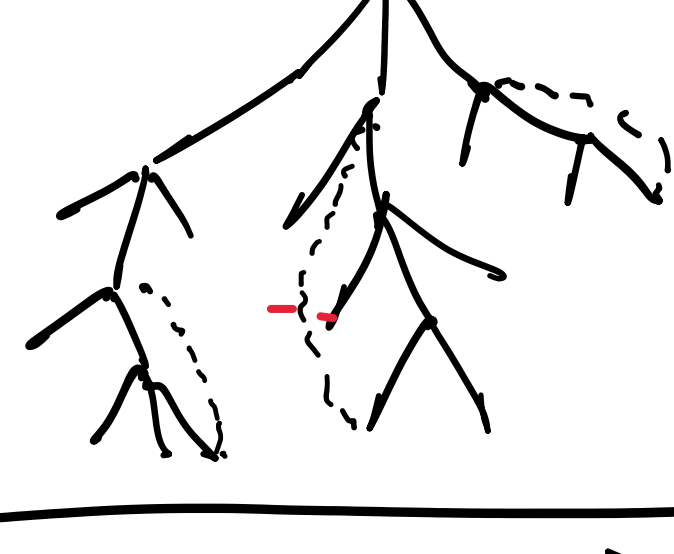
A graph theoretic characterization (Mathematical Thinking).

A vertex v is a cut vertex of a graph G iff

$\exists$ a pair of vertices $x, y \ni$ every path between x and y contains v .

(Computationally infeasible).

Let us view G in terms of tree edges and non tree edges with respect to some DFS done on G .



$u \cdot cv = 0$ - NOT a cut vertex
 $= 1$

Removal of u disconnects v_1 , but does not disconnect v_2 or v_3 .

$$\begin{aligned} \text{Low}(u) &= \text{Min} \{ u.d \mid u \text{ is a vertex reachable from } u \text{ by using zero or more tree edges and one back edge} \} \\ &= \text{Min} \{ u.d, \text{Min} \{ u.d \mid (u, v) \text{ is a back edge} \}, \text{Min} \{ u.d \mid v \text{ is reachable from a descendant of } u \text{ by a back edge} \} \} \end{aligned}$$

Observe that

$$\text{Min} \{ u.d \mid v \text{ is reachable from a descendant of } u \text{ by a back edge} \}$$

$$= \text{Min} \{ \text{Low}(v) \mid v \text{ is a child of } u \text{ in the DFS tree} \}.$$

Note further that all values needed to compute $\text{Low}(u)$ are not available in one place to work on them. The values mentioned in the equation are generated at various times during the execution of the program. Hence, we update $\text{Low}(u)$ as and when a relevant value for computing $\text{Low}(u)$ is generated.

Specifically, $\text{Low}(u)$ may be initialised as $u.d$ when $u.d$ was assigned a value.

① $\text{Low}(u)$ may be updated as

$$\text{Low}(u) = \text{Min} \{ \text{Low}(u), u.d \}$$

where (u, v) is a back edge and this may be done when the back edge (u, v) was encountered in the DFS.

② $\text{Low}(v)$ for a child v of u can be recursively computed and

$$\text{Low}(u) \text{ is updated as } \text{Min} \{ \text{Low}(u), \text{Low}(v) \}$$

when the control returns from v to u after finishing the DFS at the child v .

In order to carry out these steps,

we add,

$$u.\text{low} = u.d \text{ at control point ①}$$

we add,

$$u.\text{low} = \text{Min}(u.\text{low}, v.\text{low}) \text{ at control point ②, where we are backing from } v \text{ to } u$$

and finally

we add

$$u.\text{low} = \text{Min}(u.\text{low}, u.d) \text{ at control point ③ where we know that } (u, v) \text{ is a back edge.}$$

The full details are presented in the following pseudocode.

DFS-LOW(G) $\forall G$ is connected.

For each $u \in V$

$u.\text{color} = \text{white};$

$u.p = \text{NIL};$

$u.d = 0$

$u.\text{low} = 0$

Time = 0

DFS-LOW(G, u) $\forall u \in V$ is arbitrary.

DFS-LOW(G, u).

Time = Time + 1;

$u.d = \text{Time};$

$u.\text{low} = u.d$

$u.\text{color} = \text{Gray}.$

For each $v \in \text{Adj}(u)$

if ($v.\text{color} == \text{white}$)

DFS-LOW(G, v)

$u.\text{low} = \text{Min} \{ u.\text{low}, v.\text{low} \}$

else if ($v \neq u.p$)

$u.\text{low} = \text{Min} \{ u.\text{low}, v.d \}$

Time = Time + 1;

$u.p = \text{Time}$

$u.\text{color} = \text{Black}.$

end;

The Low values are useful in arriving at a new characterization for cut vertices.

Let u be a node that not the root of the DFS Tree.

The vertex u is a cut vertex iff $\exists$ a child v of u in the DFS Tree such that

$$\text{low}(v) \geq u.d$$

Root of the DFS Tree is a cut vertex iff it has more than one child in the DFS Tree.

Thus, at control point ④ we do the following minor change.

$u.cv = 1$ if u is a cut vertex and

$= 0$ if u is NOT a cut vertex.

Initially, $u.cv = 0 \forall u \in V$.

All we need to do now is add the following at control point ④.

if (u is not the root) and ($v.\text{low} \geq u.d$)
 $u.cv = 1.$

(For root node, we check if its degree is ≥ 2 in DFS tree to determine if it is a cut vertex).

Iterative version: